

Backend automated test suite

Calculation logic & tenant isolation

Repo: wergonic-django-backend · Split into sub-tasks · 5 August 2026

~510

TOTAL TEST CASES

~205

P0 MUST-SHIP

22

KNOWN BUGS COVERED

42

MODULES COVERED

Why this exists

The backend has about 17 real test methods across 14 apps — most `tests.py` files are empty stubs — and no test infrastructure. CI runs `manage.py test`, finds almost nothing and reports green. So "green" today means **nothing is tested**, not "nothing is broken".

Meanwhile the two July audits found real calculation errors and tenant-isolation defects (cross-org data leaks). This epic builds a real suite, starting from the code we already know is fragile.

The scope below comes from going through all 42 logic-bearing modules and cross-referencing both audits (35 findings, 22 of them unit-testable). Every bug site listed was checked in the source.

Goal

- Stand up backend test infrastructure — there is none today.
- Lock down the known calculation and permission defects with tests, so they can't come back while we fix them.
- Get a baseline suite that CI actually gates on, at the level the Flutter repo already runs at (78 files, ~498 tests).

Definition of done — per sub-task

- The module has tests for its main path plus the specific edge and boundary cases listed.
- Each mapped bug has a test that **fails on the current code and passes once it's fixed**.
- The suite runs green in CI, wired into the GitHub Actions workflow.
- No test touches the staging or production DB, Firebase, or live data. Tests run on isolated test data only.

How to read the scope

The **cases** column is a rough count of logical test cases per target — a relative sizing signal, not a schedule. Sequencing and estimation are yours. Phase 1 and 2 (~205 cases) are the must-ship: together they cover every known defect. The **ref** column points back to the source finding — A# is the pre-production audit (20 July), P# is the calculation-issues review.

READ THIS BEFORE WRITING EXPECTED VALUES

Four rules that decide whether your assertions are right

- Head backward bending is -5° , trunk backward bending is -10° . They are different on purpose — never assume they share a threshold. Assert them side by side so nobody "harmonises" them later.
- The -5° head flip still has to happen in one place: the risk verdict in `risk_assessment_posture` and the Word report (item 2.18), which still use -10° . `ramp_calculator` already uses -5° correctly — it only has stale `gt_10` labels to rename.
- `calculate_postures_2` emitting a 1.5 score is a real bug (item 1.2). Write the test to reproduce it red first.
- The overlapping wording in `posture_thresholds` is intentional — test the tables as pure logic, don't "fix" them.

Sub-task 0 · Test infrastructure

prerequisite

Nothing below can run without this. There is only a bare test runner today and no reusable fixtures. The approach and tooling are your call.

- A working test-run setup that CI executes and gates on — today's CI step is near-empty and always green.
- Reusable fixtures/factories for the core models the tests need: `User`, `Organization`, `UserOrganization`, `Session`, `Measurement`, `MeasurementData`, `Event`, `Category`, `Label`, `SessionActivity`, `SessionCondition`, `WorkCycleEvent`, `RAMPAssessment`, `SessionStats`, `Ticket`, `Notification`.

Phase 1 · Pure calculation logic

P0 · ~42 cases · do first

Best value per hour: pure functions, called with dicts, lists and DataFrames, no DB. Kills the worst calculation findings fast.

#	MODULE	CASES	REF	KEY CASES
1.1	<code>mec_mapping.py</code> <code>calculate_time</code>	8	—	Generic band accumulator, threshold-agnostic — no $-5/-10$ baked in, callers pass the threshold. Upper-inclusive ($v \leq \text{upper}$), lower-exclusive ($v > \text{lower}$): for band $(20, 45]$, 20 is out, 45 is in, 45.1 is out. <code>side_bending</code> uses <code>abs()</code> ; <code>elevation_angle</code> drops $v < 0$; percentages are time-weighted (<code>matching_ms / valid_ms</code>); empty input $\rightarrow \{0, 0, 0\}$.
1.2	<code>mec_mapping.py</code> <code>calculate_postures_1..6</code>	18	P8	REAL BUG <code>calculate_postures_2</code> emits 1.5 for input band $[1, 6.25]$. Before the fix: reproduce the 1.5 (red). After: never returns 1.5 and scores stay monotonic in %. Don't hard-code the new band values until the fix is defined. Also: <code>None</code> $\rightarrow$ <code>None</code> ; clamp $<0 \rightarrow 0$ and $>100 \rightarrow$ top; ladders 1, 3, 4, 5, 6 never emit 1.5.
1.3	<code>mec_segments.py</code> <code>scale_results_with_takt_time</code> , <code>get_feedback_frequency</code>	6	A15	<code>takt=60</code> , <code>count=10</code> , <code>static=120</code> $\rightarrow$ <code>600 / 120</code> . <code>takt</code> ≤ 0 raises. Frequency: metadata <code>None</code> $\rightarrow$ 52 for hand, 13 otherwise; arm variants fall through to the standard branch.
1.4	<code>posture_thresholds.py</code> <code>3 functions</code>	10	P16	Pure threshold tables that feed every report. The overlapping wording is intentional — test as-is, don't change it. Each boundary lands in exactly one band; ranges end at $(100, 'high')$; integers render without a trailing <code>.0</code> .

The cross-org data-leak blockers. Test the shared gate *and* its callers — the audit fixes all delegate to a couple of shared functions, which are the real single points of failure.

#	MODULE	CASES	REF	NOTES
2.1	users/scoping.py <code>get_request_organization</code>	8	A1 A3 A5 A6 A7	START HERE About 15 endpoints delegate to this one function. X-Client-Platform: web → consulting org vs active org; <code>_cached_org</code> memoization; unauthenticated short-circuit. If this returns the wrong org, every "fixed" view silently breaks while its own test still passes.
2.2	user_sessions/permissions.py 3 classes	12	A1	The gate that A1 assumes is "already guarded" but which has never been tested. LATENT BUG <code>SessionCrudPermission</code> calls <code>session.worker.user → AttributeError</code> when worker is None (same root cause as A18). Give it a dedicated case.
2.3	work_tags/permissions.py 4 classes	18	A1 A3	Org-admin and ergonomist gates; cross-org → False; <code>is_deactivated=True → False</code> ; swagger bypass behaviour.
2.4	users/permissions/ <code>level_0/1/2 + userCrud</code>	20	—	The role-based CRUD permission matrix.
2.5	work_tags/views.py <code>Event / Category / Label</code>	12	A1 A3	Via <code>APIClient.org_id</code> and <code>category_id</code> from the URL must never be trusted over the caller's own org.
2.6	tickets/views.py	6	A4	Integration level: tickets scoped to the org, no global IDOR.
2.7	stats_view.py <code>GenerateGeminiAnalysisView.get</code>	5	A5	The AI report GET rejects another org's session.
2.8	work_library/views.py + <code>serializers.py</code>	6	A6	An update can't re-point a record to another org's activity.
2.9	workcycles/views.py <code>TaskViewSet.get_queryset</code>	5	A7	Org-less orphan tasks are not exposed across companies.
2.10	ramp_calculator.py <code>run_ramp_calculations</code>	10	P2 A16	Head backward already uses <code>upper_threshold = -5</code> — assert -5, not -10. After the rename the payload carries <code>postures_2_...gt_5_degrees_*</code> keys, not <code>gt_10</code> . Trunk posture 4 uses -5 and doesn't change. This function must emit no -10 value at all — the -10 trunk verdict lives in 2.18, keep them from getting mixed up. Task-scoped posture 6 uses the task window, not the whole session (A16).
2.11	mec_segments.py <code>compute_mec_for_segment</code>	8	A15	DB-backed. Trunk with side-bending present but forward-bending null → no swallowed <code>UnboundLocalError</code> , and the missing-limb flags get set.
2.12	management/commands/ <code>session_check.py</code>	5	A18	The per-minute auto-finish doesn't crash on sessions whose worker was deleted.
2.13	management/commands/ <code>archive_sessions.py</code>	6	A19	Auto-archive uses the post-rename status value.

#	MODULE	CASES	REF	NOTES
2.14	notifications/ models.py + views.py	6	A20	A failed-measurement notification is filed under the session's org, not the recipient's currently active org.
2.15	stats_view.py single-session Excel export	5	A17	Head values are not written over the trunk columns.
2.16	measurements.py merge_measurment_files	9	A11	REGRESSION GUARD Already fixed — the tests keep it fixed. Merged zip includes _metadata_merged.json, the highest App.Version is chosen, axis swap is correct.
2.17	session_views.py session merge	12	A10	REGRESSION GUARD Already fixed. Measurements are rebuilt from the combined CSV, not moved. Sized higher than the rest because it needs the full async merge → reprocess pipeline.
2.18	risk_assessment_posture.py calculate_risk_posture_stats + session_docx_report.py	10	P2 P19	THE -5° FLIP This is where the head threshold actually changes. Head backward risk verdict goes from -10° to -5° (mask ≤ -5: a head row at -6° now counts; the id and label say -5°). The trunk verdict stays at -10° — trunk -7° doesn't count, -10° and -12° do. Assert head and trunk side by side. In the Word report the head row is labelled -5° and the trunk row -10°.

Phase 3 · Important logic

P1 · ~199 cases · backlog

Not covered by the audits, medium risk: filters, report maths, signal-processing helpers. Grouped counts here — ask if you want the per-module breakdown.

AREA	MODULES	CASES
Signal processing	mec_mapping.py — peak detection, static periods, butter filter, arm postures, zones, hourly	22
Wrist analysis	wrist_mec_mapping.py — position, custom zones, peaks, static	24
Report maths	report_service.py — label-bounds parsing, bin midpoints, weighted mean/median, option and velocity blocks	46
Distance zones	transformations.py :: compute_distance_zone_minutes plus the round/timezone helpers — the pure banding that feeds posture 6, i.e. the numeric core behind A16	8
Pie & risk stats	pie_stats.py, risk_assessment_posture.py — P18/P19 banding. Guard the intentional constants; the head -5° flip itself is P0, see 2.18	24
Data reshaping	stats.py, pure helpers in measurements.py, task_measurments.py, filters.py :: pair_measurements_by_time, preprocessing and swapping	58
DOCX helpers	session_docx_report.py :: get_risk_image, format_duration_seconds	11

Phase 4 · Nice to have

P2 · ~110 cases · backlog

Lower risk or harder to test. The TNO pipeline, including extractTaskCharacteristics, sits here — it's deliberately parked for now. Also: inversion correction, time bounds, percentage change, swapped pie stats, user-service claims, report JSON and Excel value maps, the work_tags event upsert, and firebase_auth/authentication.py ::

authenticate (the token-parsing branch — security-relevant, it silently returns None on an unknown uid). The report-formatting findings P7 and P17 are also in here.

Out of scope

don't spend time here

-
- Raw-SQL percentile paths in `stats.py` — Postgres/Timescale-only and would need huge fixtures. The reusable zone and pivot logic is covered elsewhere.
 - Firebase and DB IO orchestration in `measurements.py`, `task_measurments.py`, `calculation_service.py` — the underlying maths is covered by the pure targets.
 - DOCX/XLSX byte assembly (`build_session_word_report`, the Excel builders, `docx_pie_charts.py`) — the data-selection branching is covered through the helpers.
 - Firebase Storage pass-throughs (`tickets/utls.py`, `devices/utls.py`) and user-service create/update/delete — heavy multi-system mocking for little return.
-